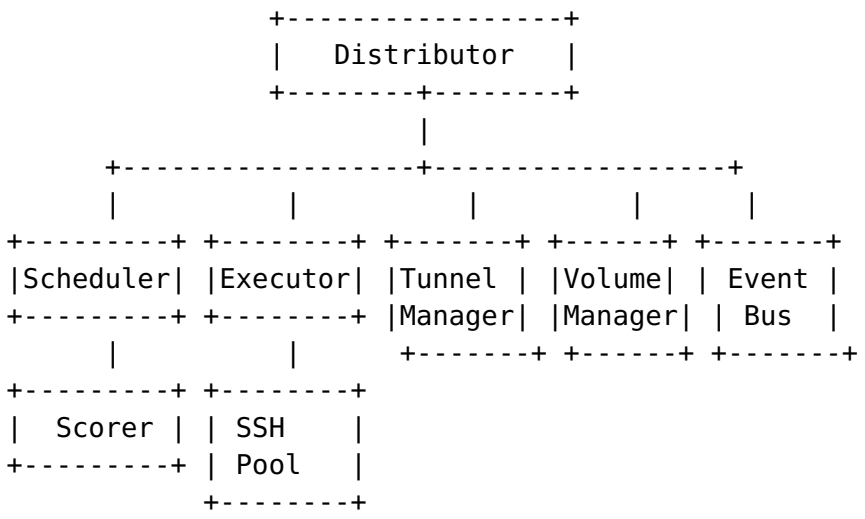
REMOTE_DISTRIBUTION

Remote Distribution Guide

Overview

The Containers module supports distributing containers across local and remote hosts via SSH. This guide covers configuration, scheduling strategies, networking, volume sharing, and failover.

Architecture



The `Distributor` composes:

- **Scheduler** — selects hosts using resource-aware scoring
- **RemoteExecutor** — executes commands via SSH with ControlMaster pooling
- **TunnelManager** — creates SSH tunnels for cross-host networking
- **VolumeManager** — mounts local volumes on remote hosts
- **HostManager** — registry of remote hosts with resource probing

Quick Start

1. Configure Remote Hosts

Copy `.env.example` to `.env` and define your remote hosts:

```
CONTAINERS_REMOTE_ENABLED=true
CONTAINERS_REMOTE_SCHEDULER=resource_aware

CONTAINERS_REMOTE_HOST_1_NAME=gpu-server
CONTAINERS_REMOTE_HOST_1_ADDRESS=192.168.1.100
CONTAINERS_REMOTE_HOST_1_PORT=22
CONTAINERS_REMOTE_HOST_1_USER=deploy
CONTAINERS_REMOTE_HOST_1_KEY=~/.ssh/gpu_key
CONTAINERS_REMOTE_HOST_1_RUNTIME=docker
CONTAINERS_REMOTE_HOST_1_LABELS=gpu=true,arch=amd64
```

2. Load Configuration

```
cfg, err := envconfig.LoadFromEnv()
if err != nil {
    log.Fatal(err)
}
hosts := cfg.ToRemoteHosts()
```

3. Create Components

```
opts := remote.DefaultOptions()
executor := remote.NewSSHExecutor(opts)
hm := remote.NewDefaultHostManager(opts)
for _, h := range hosts {
    hm.AddHost(h)
}

sched := scheduler.NewDefaultScheduler(hm,
    scheduler.WithStrategy(scheduler.StrategyResourceAware),
)

dist := distribution.NewDistributor(
    distribution.WithScheduler(sched),
    distribution.WithHostManager(hm),
    distribution.WithExecutor(executor),
)
```

4. Distribute Containers

```
summary, err := dist.Distribute(ctx,
    []scheduler.ContainerRequirements{
```

```
        {Name: "web", Image: "nginx:latest", CPUCores: 2,
        MemoryMB: 512},
        {Name: "api", Image: "myapp:latest", CPUCores: 4,
        MemoryMB: 1024},
        {Name: "cache", Image: "redis:latest", CPUCores: 1,
        MemoryMB: 256},
    },
)
```

Scheduling Strategies

Strategy	Description	Best For
`resource_aware`	Scores hosts by weighted CPU/memory/disk/network availability	General use (default)
`round_robin`	Distributes evenly across hosts in order	Uniform workloads
`affinity`	Prefers hosts matching container labels	GPU/specialized hardware
`spread`	Minimizes per-host container density	High availability
`bin_pack`	Fills most-utilized hosts first	Cost optimization

Scoring Weights (resource_aware)

Resource	Weight	Description
CPU	40%	Available CPU percentage
Memory	40%	Available memory percentage
Disk	10%	Available disk percentage
Network	10%	Available network bandwidth

Configurable via `scheduler.WithCPUWeight()`, `WithMemoryWeight()`, etc.

Resource Reserve

By default, 10% of each host's resources are reserved (not allocated to containers). Configure with `scheduler.WithReservePercent()`.

SSH Connection Pooling

The `SSHExecutor` uses SSH ControlMaster sockets for connection multiplexing:

- First connection to a host establishes a ControlMaster socket
- Subsequent commands reuse the socket (no SSH handshake overhead)
- Connections auto-close after `ControlPersist` seconds of inactivity
- Thread-safe with reference counting

Configure via:

```
opts := remote.WithControlMasterEnabled(true)
opts = remote.WithControlPersist(300) // 5 minutes
opts = remote.WithMaxConnections(10)
```

SSH Tunnels

Create local port forwarding (remote container ports appear local):

```
tm := network.NewDefaultTunnelManager(hm, executor)
info, err := tm.CreateTunnel(ctx, "gpu-server",
    network.TunnelSpec{
        Direction:  network.TunnelLocal,
        RemoteHost: "localhost",
        RemotePort: "8080",
    },
)
// Access remote container at localhost:{info.LocalPort}
```

Create reverse forwarding (local services accessible from remote):

```
info, err := tm.CreateTunnel(ctx, "gpu-server",
    network.TunnelSpec{
        Direction:  network.TunnelRemote,
        LocalPort:  "5432",
        RemoteHost: "localhost",
        RemotePort: "5432",
    },
)
// Remote containers can access local PostgreSQL at localhost:5432
```

Volume Sharing

SSHFS (Real-time, bidirectional)

```
vm := volume.NewDefaultVolumeManager(executor)
err := vm.Mount(ctx, volume.VolumeMount{
    Name:       "app-data",
    HostName:   "gpu-server",
    LocalPath:  "/data/app",
    RemotePath: "/mnt/app-data",
    Type:       volume.MountSSHFS,
})
```

NFS (High performance, read-heavy)

```
err := vm.Mount(ctx, volume.VolumeMount{
    Name:       "shared-models",
    HostName:   "gpu-server",
    LocalPath:  "/data/models",
    RemotePath: "/mnt/models",
    Type:       volume.MountNFS,
})
```

rsync (Periodic sync, one-way)

```
err := vm.Mount(ctx, volume.VolumeMount{
    Name:       "config-sync",
    HostName:   "gpu-server",
    LocalPath:  "/etc/myapp",
    RemotePath: "/etc/myapp",
    Type:       volume.MountRsync,
})
// Trigger manual sync
err = vm.Sync(ctx, "config-sync")
```

Failover

The `FailoverHandler` automatically detects offline hosts and reschedules containers:

```
fh := distribution.NewFailoverHandler(dist)
actions, err := fh.CheckAndFailover(ctx)
for _, action := range actions {
    fmt.Printf("Container %s: %s -> %s (reason: %s)\n",
        action.ContainerName,
        action.OriginalHost,
        action.NewHost,
        action.Reason,
```

```
    )
}
```

Degraded Host Detection

Identify hosts that are reachable but resource-constrained:

```
snapshots := hm.ProbeAll(ctx)
degraded := distribution.DetectDegradedHosts(
    snapshots, 90.0, 90.0, // CPU and memory thresholds
)
```

Cluster Monitoring

Aggregate resource snapshots from all hosts:

```
localSnapshot := localMonitor.Snapshot()
remoteSnapshots := hm.ProbeAll(ctx)
cluster := monitor.NewClusterSnapshot(localSnapshot,
        remoteSnapshots)

fmt.Printf("Cluster: %d hosts, %d CPU cores, %d MB RAM\n",
    cluster.HostCount,
    cluster.TotalCPUCores,
    cluster.TotalMemoryMB,
)
```

Event Types

The distribution system emits 12 event types via the EventBus:

Event	Description
`remote.host.online`	Remote host came online
`remote.host.offline`	Remote host went offline
`remote.host.degraded`	Remote host is resource-constrained
`distribution.scheduled`	Containers scheduled for placement
`distribution.deployed`	Container deployed to host
`distribution.migrated`	Container migrated between hosts
`distribution.started`	Distribution workflow began
`distribution.completed`	Distribution workflow finished
`tunnel.created`	SSH tunnel established
`tunnel.closed`	SSH tunnel closed
`tunnel.failed`	SSH tunnel creation failed
`volume.mounted`	Remote volume mounted

Event	Description
`volume.unmounted`	Remote volume unmounted

7-Phase Distribution Workflow

1. **Probe** — Probe remote hosts for resource availability
2. **Schedule** — Run scheduler to determine container placement
3. **Volumes** — Mount required volumes on remote hosts
4. **Deploy** — Deploy containers (local via runtime, remote via SSH)
5. **Tunnels** — Create SSH tunnels for cross-host networking
6. **Health** — Run health checks on all deployed containers
7. **Events** — Emit distribution events

Password Auth Bootstrap

When SSH key auth isn't pre-configured on a remote host, the module can automatically bootstrap it using password authentication:

1. Connects via Go-native SSH (`golang.org/x/crypto/ssh`) with the configured password
2. Reads the local public key (e.g., `~/.ssh/id_ed25519.pub`)
3. Appends it to the remote `~/.ssh/authorized_keys`
4. Verifies key auth works via the CLI SSH executor

This happens automatically when:

- `Password` is set in the host config
- `KeyPath` is set
- CLI SSH key auth fails

```
if executor.NeedsBootstrap(ctx, host) {
    err := executor.BootstrapKeyAuth(ctx, host)
}
```

Configure via environment:

```
CONTAINERS_REMOTE_DEFAULT_SSH_PASSWORD=your-password
CONTAINERS_REMOTE_HOST_1_PASSWORD=host-specific-password
```

Real Deployment Example: thinker.local

A complete example deploying all HelixAgent infrastructure containers to a local network host.

Containers/.env

```
CONTAINERS_REMOTE_ENABLED=true
CONTAINERS_REMOTE_DEFAULT_SSH_USER=milosvasic
CONTAINERS_REMOTE_DEFAULT_SSH_KEY=~/.ssh/id_ed25519
CONTAINERS_REMOTE_DEFAULT_SSH_PASSWORD=<password>
CONTAINERS_REMOTE_DEFAULT_RUNTIME=docker
CONTAINERS_REMOTE_SCHEDULER=resource_aware
CONTAINERS_REMOTE_PORT_RANGE_START=20000
CONTAINERS_REMOTE_PORT_RANGE_END=30000
CONTAINERS_REMOTE_VOLUME_TYPE=rsync
CONTAINERS_REMOTE_CONNECT_TIMEOUT=10
CONTAINERS_REMOTE_COMMAND_TIMEOUT=120
CONTAINERS_REMOTE_SSH_CONTROL_MASTER=true
CONTAINERS_REMOTE_SSH_CONTROL_PERSIST=300
CONTAINERS_REMOTE_SSH_MAX_CONNECTIONS=10

CONTAINERS_REMOTE_HOST_1_NAME=thinker
CONTAINERS_REMOTE_HOST_1_ADDRESS=thinker.local
CONTAINERS_REMOTE_HOST_1_PORT=22
CONTAINERS_REMOTE_HOST_1_USER=milosvasic
CONTAINERS_REMOTE_HOST_1_KEY=~/.ssh/id_ed25519
CONTAINERS_REMOTE_HOST_1_PASSWORD=<password>
CONTAINERS_REMOTE_HOST_1_RUNTIME=docker
CONTAINERS_REMOTE_HOST_1_LABELS=role=infrastructure,env=development
```

HelixAgent .env

Add `SVC_*` overrides pointing services to the remote host:

```
DB_HOST=thinker.local
REDIS_HOST=thinker.local
SVC_POSTGRESQL_HOST=thinker.local
SVC_POSTGRESQL_REMOTE=true
SVC_REDIS_HOST=thinker.local
SVC_REDIS_REMOTE=true
SVC_CHROMADB_HOST=thinker.local
SVC_CHROMADB_REMOTE=true
SVC_QDRANT_HOST=thinker.local
SVC_QDRANT_REMOTE=true
```

Also duplicate the `CONTAINERS_REMOTE_*` vars so they're available at HelixAgent startup.

What Happens on Startup

1. `NewAdapterFromConfig()` loads `Containers/.env`
2. Detects `CONTAINERS_REMOTE_ENABLED=true`
3. For each host, checks if key auth works (`NeedsBootstrap`)
4. If not, bootstraps key auth using the configured password
5. Registers hosts in the HostManager

6. Infrastructure functions (`ensureLSPServers`, `ensureRAGServices`, `ensureMCPServers`, `ensureRequiredContainersWithConfig`) detect `RemoteEnabled()` and call `RemoteComposeUp()` instead of local `ComposeUp()`
7. Compose files are copied to `/opt/helixagent/` on the remote host
8. `docker compose up -d` is run remotely via SSH

Verification

```
ssh milosvasic@thinker.local echo ok
ssh milosvasic@thinker.local docker ps
curl http://localhost:7061/health
```

Environment Variables

See `.env.example` for the complete reference. Key variables:

Variable	Default	Description
`CONTAINERS_REMOTE_ENABLED`	`false`	Enable remote distribution
`CONTAINERS_REMOTE_SCHEDULER`	`resource_aware`	Scheduling strategy
`CONTAINERS_REMOTE_DEFAULT_SSH_USER`	—	Default SSH user
`CONTAINERS_REMOTE_DEFAULT_SSH_KEY`	—	Default SSH key path
`CONTAINERS_REMOTE_DEFAULT_SSH_PASSWORD`	—	Default SSH password (bootstrap)
`CONTAINERS_REMOTE_DEFAULT_RUNTIME`	`docker`	Default container runtime
`CONTAINERS_REMOTE_PORT_RANGE_START`	`20000`	Tunnel port range start
`CONTAINERS_REMOTE_PORT_RANGE_END`	`30000`	Tunnel port range end
`CONTAINERS_REMOTE_VOLUME_TYPE`	`sshfs`	Default volume type
`CONTAINERS_REMOTE_CONNECT_TIMEOUT`	`10`	SSH connect timeout (seconds)
`CONTAINERS_REMOTE_COMMAND_TIMEOUT`	`120`	SSH command timeout (seconds)
`CONTAINERS_REMOTE_SSH_CONTROL_MASTER`	`true`	Enable ControlMaster pooling
`CONTAINERS_REMOTE_SSH_CONTROL_PERSIST`	`300`	ControlMaster persist (seconds)
`CONTAINERS_REMOTE_SSH_MAX_CONNECTIONS`	`10`	Max concurrent SSH connections
`CONTAINERS_REMOTE_HOST_N_NAME`	—	Host N name
`CONTAINERS_REMOTE_HOST_N_ADDRESS`	—	Host N address

Variable	Default	Description
CONTAINERS_REMOTE_HOST_N_PORT	22	Host N SSH port
CONTAINERS_REMOTE_HOST_N_USER	—	Host N SSH user
CONTAINERS_REMOTE_HOST_N_KEY	—	Host N SSH key path
CONTAINERS_REMOTE_HOST_N_PASSWORD	—	Host N SSH password
CONTAINERS_REMOTE_HOST_N_RUNTIME	—	Host N container runtime
CONTAINERS_REMOTE_HOST_N_LABELS	—	Host N labels (key=value,key=value)